

LAB-09 단순선형회귀(Simple Linear Regression) — 기본기 정리

이 문서는 "회귀분석을 처음 볼 때마다 여기부터 다시 본다"는 기준으로 만든 참고 문서다. 다른 데이터셋으로 회귀분석을 할 때도 이 순서와 개념 체크리스트를 그대로 재사용하면 된다.

0. 오늘 실습에서 확인해야 할 전체 그림

오늘 코드는 같은 것을 두 가지 방식으로 반복해서 보여주는 구조다. 이 구조 자체가 핵심 학습 포인트다.

1. 원본 라이브러리(statsmodels)를 직접 써서 회귀분석하는 버전 → #03, #04
2. 내가 만든 모듈(helpers 패키지의 my_ols)로 감싸서 같은 걸 한 줄로 하는 버전 → #05

즉, "raw 코드로 원리를 이해 → 모듈로 감싸서 재사용"이라는 흐름이다. 이걸 앞으로 어떤 분석을 하든 똑같이 적용되는 사고방식이니 아래에서 따로 정리한다.

전체 분석 순서(재사용 템플릿):

- 1) 데이터 로드 & 확인 (head/tail)
- 2) EDA: 산점도 + 상관분석 (선형관계가 있는지, 회귀를 써도 되는지 먼저 검증)
- 3) 회귀모형 적합 (fit)
- 4) 결과 해석 (summary: 계수, p-value, R^2 , F-statistic)
- 5) 예측 (predict) - 새로운 x값 또는 미래값에 대해
- 6) 시각화로 실제값 vs 예측값 비교 (long format으로 melt 후 lmpplot)
- 7) (선택) 모듈화된 버전으로 같은 과정을 재현해서 검증

이 7단계는 회귀분석뿐 아니라 이후 다중회귀, 로지스틱회귀에도 그대로 재사용될 구조다.

1. EDA — 회귀를 하기 "전에" 반드시 확인하는 것

왜 상관분석을 먼저 하는가

회귀분석은 "두 변수 사이에 **선형** 관계가 있다"는 걸 전제로 한다. 이 전제가 깨지면 직선을 그어봤자 의미가 없다. 그래서 회귀를 적합하기 전에 **상관분석으로 선형성과 방향, 강도를 먼저 확인**하는 것이 순서다.

`my_stats.correlation()`이 자동으로 해주는 것:

- 정규성 검정 → Pearson을 쓸지 Spearman을 쓸지 자동 선택
- Ramsey RESET 검정으로 **선형성** 확인
- 영향점(influential point) 확인
- 상관계수 + 유의확률(p-value) + 강도 해석까지 한 번에

상관계수 해석 기준 (말로 풀어서)

- 절댓값 0.1 미만: 상관관계 거의 없음
- 0.1~0.3: 약한 상관
- 0.3~0.7: 중간(moderate) 상관 — 오늘 father-son 데이터의 0.5가 여기
- 0.7 이상: 강한 상관

가설검정 관점에서 본 상관분석

상관계수 하나 구했다고 끝이 아니라, **"**"이 상관관계가 우연이 아니라 진짜인지"**를 통계적으로 검증하는 것이 핵심이다.

- **귀무가설(H_0):** 모집단에서 두 변수의 상관계수 ρ (로우, 모상관계수)는 0이다. 즉, 두 변수는 선형적으로 아무 관계가 없다.
 - 기호로: $H_0: \rho = 0$
- **대립가설(H_1):** 모상관계수 ρ 는 0이 아니다. 즉, 두 변수 사이에 선형관계가 존재한다.
 - 기호로: $H_1: \rho \neq 0$
- 결과에서 **significant=True**이고 $p < 0.05$ 라면, "우연히 이 정도 상관이 나올 확률이 5% 미만"이라는 뜻이므로 **귀무가설을 기각**하고 "유의한 상관관계가 있다"고 결론 내린다.

면접에서 말로 풀면: "표본에서 관찰된 상관계수가 우연이 아니라 모집단에서도 실제로 존재하는 관계인지를 확인하기 위해, 모상관계수가 0이라는 귀무가설을 세우고 검정했습니다. p값이 유의수준 0.05보다 작아 귀무가설을 기각했고, 두 변수 간 유의한 양의 상관관계가 있다고 판단했습니다."

2. 단순선형회귀 모형 — 수식과 의미

모형식

$$y = \beta_0 + \beta_1 x + \varepsilon$$

말로 풀면:

- y : 종속변수(예측하고 싶은 결과값, 오늘 예시에선 제동거리 **dist**, 아들 키 **sheight**)
- x : 독립변수(원인/설명변수, **speed**, **fheight**)
- β_0 (베타 제로): **절편(intercept)** — x 가 0일 때 y 의 예상값. 상수항.
- β_1 (베타 원): **기울기(slope, 회귀계수)** — x 가 1단위 증가할 때 y 가 평균적으로 얼마나 변하는지
- ε (엡실론): **오차항** — 모형이 설명하지 못하는 나머지 부분(잔차의 모집단 버전)

우리가 데이터로 추정한 결과는 이렇게 쓴다(모수 β 대신 추정치 b 또는 $\hat{\beta}$ 로):

$$\hat{y} = b_0 + b_1 x$$

오늘 father-son 예시: $\widehat{sheight} = 33.887 + 0.514 \times fheight \rightarrow$ 말로 풀면 "아버지 키가 1인치 커질 때마다 아들 키는 평균적으로 0.514인치 커진다고 예측된다."

회귀계수에 대한 가설검정 (이게 summary에 나오는 p-value의 의미)

`fit.summary()`에 나오는 **speed**나 **fheight** 옆의 p-value는 그 변수의 기울기(β_1)가 정말 0이 아닌지를 검정한 결과다.

- **귀무가설(H_0):** $\beta_1 = 0 \rightarrow$ 독립변수는 종속변수를 설명하는 데 아무 도움이 안 된다(기울기가 없다, 즉 관계가 없다).
- **대립가설(H_1):** $\beta_1 \neq 0 \rightarrow$ 독립변수는 종속변수에 유의한 영향을 준다.
- $p < 0.05$ 면 귀무가설 기각 $\rightarrow$ "이 독립변수는 통계적으로 유의미한 설명력을 가진다."

절편(intercept)에 대해서도 같은 논리로 $H_0: \beta_0 = 0$ 검정이 나오지만, 실무에서는 절편의 유의성보다 **기울기의 유의성**이 훨씬 중요하게 다뤄진다(절편은 보통 $x=0$ 이 현실적으로 의미 없는 경우가 많아서).

최소제곱법(OLS, Ordinary Least Squares) — 계수를 어떻게 구하는가

회귀직선은 "실제값과 예측값의 차이(잔차, residual)를 제공해서 합한 값이 최소가 되도록" 직선을 긋는 방식으로 구해진다.

$$\min \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

말로 풀면: 모든 점에서 직선까지의 수직 거리(오차)를 제공해서 다 더한 값이 가장 작아지는 직선을 "가장 잘 맞는 직선"으로 정의한다는 것. 제곱을 하는 이유는 (+)오차와 (-)오차가 서로 상쇄되는 걸 막고, 큰 오차에 더 큰 페널티를 주기 위함이다.

결정계수 R^2 (R-squared)

모형이 종속변수의 변동을 얼마나 설명하는지를 0~1 사이 값으로 나타낸 것.

- $R^2 = 0.5$ 라면 "독립변수가 종속변수 변동의 50%를 설명한다"는 뜻.
- 나머지 50%는 모형이 설명하지 못하는, 오차항(다른 요인들, 랜덤한 변동)에 해당한다.

F-통계량 (모형 전체의 유의성 검정)

summary 맨 위쪽에 나오는 F-statistic은 "이 회귀모형 전체가 통계적으로 의미가 있는가"를 검정한다.

- 단순선형회귀(독립변수 1개)에서는 F검정 결과와 기울기의 t검정 결과가 사실상 같은 결론을 준다(변수가 하나뿐이니까). 하지만 **다중회귀로 넘어가면 F검정(모형 전체)과 개별 t검정(계수 하나하나)은 별개의 질문**이 되므로 이 구분을 지금부터 익혀두는 게 좋다.
 - H_0 (F검정): 모든 독립변수의 회귀계수가 동시에 0이다 (모형이 아무것도 설명 못한다)
 - H_1 (F검정): 적어도 하나의 회귀계수는 0이 아니다

3. 코드 단계별 의미 (raw statsmodels 버전 기준)

3-1. 독립변수 행렬 준비 + 상수항 추가

```
x = origin[["speed"]]
x_input = add_constant(x)
```

`add_constant()`를 반드시 해줘야 하는 이유: statsmodels의 `OLS`는 절편(β_0)을 자동으로 넣어주지 않는다. 상수항(값이 전부 1인 열)을 직접 추가해줘야 절편이 추정된다. 이걸 빼먹으면 "원점을 지나는 직선"으로 강제로 적합되어버려서 결과가 완전히 달라진다. (반면 `formula_ols("dist ~ speed", ...)`처럼 formula API를 쓰면 절편이 자동으로 포함된다 — 이 차이를 꼭 기억해야 한다.)

3-2. 모델 객체 생성과 적합(fit)의 차이

```
model = OLS(y, x_input)    # 아직 학습 안 됨, '모형의 틀'만 생성
fit = model.fit()          # 실제로 데이터를 이용해  $\beta_0, \beta_1$ 을 추정
```

`model`은 "이런 식으로 회귀분석을 하겠다"는 설계도이고, `fit`은 그 설계도에 실제 데이터를 넣어 계수를 계산해낸 결과 객체다. `predict()`는 `model`이 아니라 `fit`에서 호출해야 한다(계수가 있어야 예측이 가능하니까).

3-3. formula API 방식

```
model = formula_ols("dist ~ speed", data=origin)
```

"`y ~ x`" 형태의 R 스타일 수식으로 표현. 상수항이 자동 포함되고, 컬럼명을 그대로 쓸 수 있어서 행렬 방식보다 코드가 짧다. 변수가 많아질수록("`y ~ x1 + x2 + x3`") 이 방식이 훨씬 편해진다.

3-4. 예측(predict)할 때 주의점

```
predict_input = add_constant(df["speed"])
df['pred'] = fit.predict(predict_input)
```

예측할 때 넣는 입력 데이터도 학습할 때와 똑같은 형태여야 한다. 학습 시 상수항을 추가했으면 예측 시에도 상수항을 추가해야 한다. 이걸 형태가 안 맞으면 에러가 나거나 엉뚱한 값이 나오는 아주 흔한 실수 포인트다.

3-5. 시각화 전 melt로 long format 변환

```
df_melt = df.melt(id_vars="speed", value_vars=["dist", "pred"],
var_name="variable", value_name="value")
```

`dist`(실제값)와 `pred`(예측값)를 각각 별도 컬럼(wide format)으로 두면 seaborn에서 `hue`로 구분해서 그리기 어렵다. `melt`로 "실제/예측" 구분을 하나의 범주형 컬럼(`variable`)으로 만들어야 `hue='variable'` 한 줄로 두 종류를 색깔 구분해서 겹쳐 그릴 수 있다. → **이건 pandas 기본기 preference랑도 연결되는 부분: `melt`는 원본을 바꾸지 않고 새 DataFrame을 반환하므로 `df_melt = df.melt(...)`처럼 반드시 재할당해야 한다.**

4. 모듈화 버전(`my_ols`) — "직접 코딩 vs 모듈 호출"을 어떻게 사고할 것인가

```
fit = my_ols.fit_model(origin, y='dist', summary=True)
pred = my_ols.predict(fit, origin[['speed']])
```

핵심 사고방식: "raw 코드가 먼저, 모듈은 그걸 감싼 것"

`my_ols.fit_model()` 내부에는 결국 위에서 본 `add_constant + OLS/formula_ols + .fit()` 로직이 그대로 들어있다고 생각하면 된다. 즉 모듈 함수는 **매번 반복하는 5~6줄짜리 정형화된 코드를 함수 하나로 감싸놓은 것뿐이다.** 그래서:

1. 원리는 **raw 코드(3장)로 이해한다.** 상수항을 왜 추가하는지, `fit`과 `model`이 왜 분리되는지 등은 모듈 뒤에 숨어있어도 반드시 알고 있어야 한다. 그래야 모듈이 이상하게 동작할 때 "내부에서 뭐가 잘못됐는지" 짐작할 수 있다.
2. 모듈은 **"같은 작업을 여러 데이터셋/여러 실습에서 반복할 때" 쓴다.** 오늘처럼 cars 데이터로 한 번, father-son 데이터로 한 번 — 똑같은 흐름을 두 번 타는 게 바로 모듈화가 필

요한 신호다.

3. 모듈 함수를 만들 때는 원본 라이브러리 함수를 참고 삼아, 매개변수(파라미터)를 내가 자주 바꾸는 값 위주로 노출시킨다. 예: `fit_model(df, y=..., summary=True/False)` 처럼 "종속변수 이름"과 "요약 출력 여부"만 바뀌가며 쓸 수 있게.

모듈을 확장할 때 실천 순서 (앞으로 계속 쓸 흐름)

- 1) 새로운 기능이 필요하다고 느낀 시점에, 먼저 raw 코드로 "한 번" 직접 짜본다.
- 2) 그 코드가 의도대로 동작하는지 작은 데이터로 검증한다 (전체를 다 짜고 나서 확인 x).
- 3) 동작이 확인되면, 그 로직을 함수로 뽑아서 `my_ols.py` 같은 모듈 파일에 넣는다.
- 4) `__init__.py`에서 import 경로를 연결한다 (`from . import my_ols` 등).
- 5) 노트북에서는 `importlib.reload(helpers)`로 리로드하며 바로바로 확인한다.
- 6) 기존 raw 버전 결과와 모듈 버전 결과가 "값이 똑같이 나오는지" 대조해서 검증한다.

→ 오늘 코드의 #05가 정확히 이 6단계 중 5~6번 단계를 보여주는 것: cars 데이터로 raw 버전 결과와 모듈 버전 결과를 나란히 비교해서 "모듈이 제대로 만들어졌는지"를 확인하는 셈이다.

"처음부터 끝까지 다 짠다"가 아니라 "단계마다 확인한다"

- EDA 하나 하고 → 결과가 말이 되는지 확인
- fit 하나 하고 → summary 찍어서 계수 부호/유의성이 상식과 맞는지 확인
- predict 하나 하고 → `tail()`로 몇 개 값만 눈으로 검산
- 시각화 하고 → 예측값이 회귀직선 위에 정확히 얹히는지 눈으로 확인 (오늘 미션4의 검증 포인트)

이렇게 매 단계마다 눈으로/숫자로 검증하고 다음 단계로 넘어가는 습관이 진짜 핵심이다. 다 짜놓고 마지막에 한 번에 디버깅하려고 하면 어디서 틀렸는지 찾기 어렵다.

5. 다른 데이터셋에 적용할 때 체크리스트

새로운 데이터로 단순선형회귀를 할 때 이 순서대로 점검:

- ☐ 종속변수(y)와 독립변수(x)를 명확히 정했는가? (무엇을 예측하고, 무엇으로 예측할 것인가)

- ☐ 결측치/이상치를 먼저 확인했는가? (`my_qtcheck.numerical_summary` 같은 기초 점검)
- ☐ 산점도로 시각적 선형 경향을 먼저 봤는가?
- ☐ `my_stats.correlation()`으로 상관계수·유의성·선형성 가정을 확인했는가?
- ☐ (raw 버전) `add_constant`를 잊지 않았는가? / (formula 버전) 수식 문자열의 컬럼 명이 정확한가?
- ☐ `fit.summary()`에서 계수 부호가 상식적으로 말이 되는지, p-value가 유의한지, R^2 가 얼마나 되는지 확인했는가?
- ☐ 예측 시 입력 데이터 형태(상수항 포함 여부 등)가 학습 때와 같은가?
- ☐ 실제값 vs 예측값을 시각화로 대조했는가?
- ☐ (여유가 되면) 모듈화된 함수로 같은 과정을 재현해 raw 버전과 결과가 일치하는지 검증했는가?

6. 면접 대비용 — 말로 설명하는 연습 문장

- "단순선형회귀는 하나의 독립변수로 종속변수를 예측하는 모형이며, 실제값과 예측값의 차이인 잔차 제곱합을 최소화하는 방식(최소제곱법)으로 계수를 추정합니다."
- "회귀계수의 유의성은 귀무가설을 '해당 독립변수의 회귀계수는 0이다', 즉 종속변수에 영향을 주지 않는다고 설정하고 검정합니다. p값이 유의수준보다 작으면 귀무가설을 기각하고 유의한 영향을 준다고 해석합니다."
- " R^2 는 모형이 종속변수의 총 변동 중 몇 %를 설명하는지를 나타내는 지표로, 1에 가까울수록 설명력이 높습니다."
- "회귀분석 전에 상관분석으로 두 변수 간 선형성과 상관의 유의성을 먼저 검토했습니다. 이는 회귀모형의 기본 가정인 선형성이 충족되는지를 사전에 확인하기 위함입니다."
- "상수항을 추가하지 않으면 절편이 0으로 고정된 모형이 되어버리기 때문에, 행렬 기반 방식에서는 반드시 상수항 열을 추가해줘야 합니다."

7. 이번 실습에서 pandas 관련 재확인 포인트

- `df.loc[len(df)] = [...]`: 존재하지 않는 인덱스 위치에 값을 대입하면 새로운 행이 추가된다 (행 추가 트릭).
- `df.melt()`, `.copy()`, `pd.concat()` 모두 원본을 바꾸지 않고 새 객체를 반환하므로 반드시 변수에 재할당해서 사용해야 한다 (기존 pandas 기본 원칙과 동일).

8. my_ols.py 실제 소스 뜯어보기

실제로 만든 모듈 코드를 보면, 3장에서 다룬 raw 코드와 **완전히 똑같은 5줄**이 함수 안에 그대로 들어있는 걸 확인할 수 있다. 이게 "raw 코드를 이해하고 나서 모듈을 봐야 하는 이유"를 그대로 보여준다.

```
def fit_model(data, y, summary=False):
    if y not in data.columns:
        raise KeyError(f"종속변수 '{y}'가 데이터 프레임의 컬럼에 존재하지 않습니다.")

    x = data.drop(columns=[y])          # y를 제외한 '나머지 전부'를 독립변수로
    y_series = data[y]
    x_input = add_constant(x)
    model = OLS(y_series, x_input)
    fit = model.fit()

    if summary:
        print(fit.summary())

    return fit
```

8-1. 가장 중요한 설계 차이: "지정한 x 1개"가 아니라 "y를 뺀 나머지 전부"

3장의 raw 코드에서는 `x = origin[["speed"]]`처럼 독립변수를 직접 하나씩 골라서 넣었다. 그런데 `my_ols.fit_model()`은 `x = data.drop(columns=[y])`, 즉 **y를 제외한 나머지 모든 컬럼을 자동으로 독립변수로 사용한다**.

- 오늘 실습에서는 데이터가 (speed, dist) 또는 (fheight, sheight)처럼 컬럼이 2개뿐이라 결과적으로 "독립변수 1개"가 되어 raw 코드와 값이 똑같이 나온 것뿐이다.
- 즉 이 함수는 사실 **단순선형회귀 전용이 아니라 다중선형회귀까지 자동으로 대응하는 범용 함수**로 설계되어 있다. 나중에 독립변수가 여러 개인 데이터를 넣으면 별도 수정 없이 다중회귀로 바로 확장된다 — 이게 모듈화의 장점이 실제로 드러나는 지점이다.
- **주의할 부작용**: 컬럼이 2개보다 많은 데이터프레임을 그대로 넣으면, 원치 않는 컬럼(ID, 라벨, 문자열 컬럼 등)까지 전부 독립변수로 팔려 들어간다. 예를 들어 지난 미션 4에서 만든 `merged`(원본+가상 데이터를 합치며 `source`라는 문자열 컬럼을 추가한 데이터)를 그대로 `fit_model(merged, y='sheight')`에 넣으면, `source`가 숫자가 아니라서 `add_constant/OLS` 단계에서 에러가 나거나 의도치 않은 더미 처리가 된다. → 이 함수를 쓸 때는 항상 "내가 필요한 독립변수 컬럼만 남긴 df"를 만들어서 넣어야 한다 (`data[[x1, x2, y]]`처럼 필요한 컬럼만 슬라이싱한 뒤 전달).

8-2. 방어 코드(입력 검증)

```
if y not in data.columns:
    raise KeyError(...)
```

함수 맨 앞에서 `y` 컬럼명이 실제로 존재하는지부터 확인하고, 없으면 바로 명확한 에러 메시지와 함께 멈춘다. 이런 식으로 `"""함수 초입에서 잘못된 입력을 걸러내고 의미 있는 에러를 던지는 것"""`이 모듈을 만들 때 좋은 습관이다. 나중에 다른 함수를 추가할 때도 이 패턴(파라미터 검증 → 본 로직 → 반환)을 그대로 따라가면 된다.

8-3. `predict()` 함수에서 주의해야 할 진짜 함정

```
def predict(fit, new_data):
    new_data_with_const = add_constant(new_data)
    predictions = fit.predict(new_data_with_const)
    return DataFrame(predictions, columns=["pred"])
```

여기서 초보자가 자주 걸리는 함정이 하나 있다.

`add_constant()`는 기본적으로(`has_constant='skip'`) "입력 데이터에 이미 상수(값이 전혀 변하지 않는 컬럼)가 있으면 상수항을 추가하지 않고 건너뛴다." 그런데 `new_data`가 딱 1행짜리 데이터라면, 그 한 행 안의 모든 컬럼은 값이 하나뿐이라 "변하지 않는 값"처럼 보여버린다. 이 경우 `add_constant`가 "이미 상수항이 있다"고 착각하고 실제 절편 컬럼(`const`, 값 1)을 추가하지 않을 수 있다.

→ 실무에서는 예측할 새 데이터를 여러 행으로 묶어서 넣는 습관을 들이는 게 안전하다 (오늘 실습에서도 신청자 1명이 아니라 5명을 한 번에 `DataFrame`으로 만들어 넣었던 것도 이런 이유와 맞닿아 있다). 만약 정말 1건만 예측해야 한다면, 상수항이 제대로 붙었는지 `new_data_with_const`를 직접 출력해서 `const` 컬럼이 존재하는지 확인하는 습관이 필요하다.

또 하나 눈여겨볼 점: `predictions`은 `new_data`의 인덱스를 그대로 유지한 `Series`로 나오고, 이를 그대로 `DataFrame(predictions, columns=["pred"])`으로 감싸기 때문에 반환된 예측 결과의 인덱스가 입력 데이터와 1:1로 정렬되어 있다. 그래서 오늘 코드에서 `new_df['sheight'] = pred['pred'].values`처럼 값만 뽑아 붙여도 순서가 어긋나지 않은 것이다 — 인덱스 정렬이 안 맞는 상황(예: `new_data`를 필터링/정렬한 뒤 넣는 경우)이라면 `.values`로 강제로 붙이지 말고 인덱스 기준으로 합치는 게 더 안전하다.

8-4. 오늘 배운 걸 `my_ols.py`에 적용해서 정리하면

구분	raw 코드 (3장)	<code>my_ols.py</code> (모듈)
독립변수 선택	<code>origin[["speed"]]</code> 처럼 직접 지정	<code>data.drop(columns=[y])</code> 로 자동 (나머지 전부)
상수항 추가	매번 <code>add_constant()</code> 직접 호출	함수 내부에 이미 포함
입력 검증	없음	<code>y not in data.columns</code> 체크 후 <code>KeyError</code>
summary 출력	<code>print(fit.summary())</code> 항상 실행	<code>summary=True</code> 일 때만 선택적으로 실행
예측 반환 형태	<code>Series</code> 그대로 (<code>df['pred'] = fit.predict(...)</code>)	<code>DataFrame(columns=["pred"])</code> 로 표준화

이 표가 결국 "모듈화란 무엇을 얻는 작업인가"에 대한 답이다: 반복되는 코드를 줄이는 것 + 입력 검증/에러 처리를 표준화하는 것 + 반환 형태를 일관되게 만드는 것. 앞으로 새 함수를 `helpers` 패키지에 추가할 때도 이 세 가지를 목표로 삼으면 된다.

9. 연습문제(father-son) 사고의 흐름 — "왜 이렇게 코딩했는가"

여기가 사실 제일 중요한 부분이다. 코드 자체보다 문제 문장을 읽고 → 어떤 개념을 떠올리고 → 왜 그 함수/구조를 선택했는지의 흐름을 몸에 익히는 게 목표다. 아래는 그 흐름을 미션별로 재구성한 것이다.

Mission 1. 데이터와 친해지기 (EDA)

문제 문장에서 잡아야 할 신호

"데이터와 친해지기" / "산점도를 그려 눈으로 관찰" / "회귀분석을 적용해도 되는지 근거를 대라"

이 세 문구가 곧 해야 할 일의 순서를 그대로 알려준다. 특히 마지막 문장 — "적용해도 되는지 근거를 대라" — 이 표현이 나오면 머릿속에서 자동으로 떠올려야 하는 게 "회귀분석의 전제조

건 3가지(선형성, 상관의 유의성, 영향점 여부)"다. 이게 바로 `my_stats.correlation()`을 꺼내 든 이유다.

사고 흐름

1. `df.shape, df.info()` → "행/열 개수, 결측치 없는지, dtype이 숫자형인지"부터 확인 — 이거 확인 안 하고 바로 회귀 돌리면 나중에 에러 원인 찾기 힘들다.
2. `my_qtcheck.numerical_summary(df).T` → 왜 `.T`(전치)를 했는가? 컬럼이 `fheight, sheight` 딱 2개뿐이라, 전치를 안 하면 통계량(mean, std, min, max...)이 옆으로 길게 늘어서 한눈에 안 들어온다. 전치하면 "통계량 이름"이 행으로, "변수명"이 열로 와서 표가 읽기 편해진다 → **표를 보기 좋게 만드는 것도 EDA의 일부라는 감각을 익히는 부분.**
3. 산점도로 "눈으로 먼저" 확인 → 통계 검정을 돌리기 전에 항상 시각적으로 먼저 훑어보는 습관. 숫자만 보고 판단하면 이상치나 비선형 패턴을 놓칠 수 있다.
4. `my_stats.correlation(df, x='fheight', y='sheight', ...)` → 여기서 `x, y`를 지정할 때 "어느 게 원인/설명이고 어느 게 결과인가"를 먼저 정해야 한다. 아버지 키 → 아들 키라는 방향(유전적으로 아버지가 먼저 존재, 아들 키를 설명하는 변수로 쓰겠다는 의도)을 미리 정했기 때문에 `x='fheight', y='sheight'`로 넣은 것.
5. 결과 해석 시 자동으로 떠올려야 하는 것: **"상관계수 크기 → 강도 판단 기준표", "유의 확률 → 귀무가설($p=0$) 기각 여부", "RESET 검정 → 선형성 가정 충족 여부".** 이 세 가지를 다 갖춰야 "회귀분석을 적용해도 된다"는 결론에 근거가 생긴다. 하나라도 빠지면 "그냥 상관계수만 보고 회귀를 돌렸다"는 약한 근거가 된다.

Mission 2. 회귀식 유도하기

문제 문장에서 잡아야 할 신호

"종속변수", "독립변수", "적합한", "회귀식을 유도"

사고 흐름

1. Mission 1에서 이미 "aheight가 원인, sheight가 결과" 방향을 정해줬으므로, 여기서 그 방향을 그대로 모델에 넣기만 하면 된다 — **EDA 단계의 의사결정이 모델링 단계로 그대로 이어진다는 것을 확인하는 지점.**
2. raw 코드(`add_constant + OLS`)를 또 쓰지, `my_ols.fit_model()`을 쓰지 고민해야 하는 지점인데, **이미 4장/8장에서 raw 버전과 모듈 버전이 같은 결과를 낸다는 걸 검증했기 때문에,** 이제부터는 검증된 모듈을 재사용하는 게 합리적이다. → `fit = my_ols.fit_model(origin, y='sheight', summary=True)` 한 줄로 끝나는 이유가 여기 있다. (raw 코드를 매번 다시 안 쓰는 것 자체가 "검증 후 재사용"이라는 사고 흐름을 실천하는 것.)

3. `data.drop(columns=[y])` 설계 때문에 이 함수에 `origin`을 통째로 넘겨도 안전한 이유 — father-son 데이터는 컬럼이 `fheight`, `sheight` 딱 2개뿐이라, `y='sheight'`를 빼면 자동으로 `fheight`만 남는다. 컬럼이 딱 2개일 때만 안전하다는 8-1의 주의사항이 여기서 실제로 성립하는 상황.
4. `summary()` 출력에서 `const`(절편)와 `fheight`(기울기) 계수를 찾아서 수식 $sheight = 33.887 + 0.514 \times fheight$ 형태로 옮겨 적는 것 — 이게 "회귀식을 유도하라"는 요구의 실체다. `summary` 표를 그대로 두는 게 아니라 사람이 읽을 수 있는 수식 문장으로 번역해야 한다는 뜻.

Mission 3. 미지의 5명, 아들 키 예측

문제 문장에서 잡아야 할 신호

"새 등록 신청자 5명의 아버지 키 [...]을 원본 데이터와 병합한 새 데이터셋을 만들고, [...] 예측한다"

이 한 문장 안에 두 개의 작업이 숨어있다는 걸 먼저 파악해야 한다: (1) 예측하기, (2) 원본과 병합하기. 이것 하나로 뭉뚱그려서 생각하면 코드 순서가 꼬인다.

사고 흐름

1. 예측하려면 먼저 "모델이 원하는 입력 형태"로 새 데이터를 만들어야 한다. `new_x = DataFrame({'fheight': new_fheight})` — 여기서 컬럼명을 반드시 `fheight`로 맞춰야 한다는 것이 핵심 개념이다. 학습할 때 쓴 컬럼명과 예측할 때 넣는 컬럼명이 다르면 `predict()` 내부의 `add_constant` 단계에서 정렬이 안 맞아 잘못된 값이 나오거나 에러가 난다. (3-4장에서 다룬 "학습 때와 예측 때 입력 형태를 맞춰야 한다"는 원칙이 여기서 실전으로 등장.)
2. `pred = my_ols.predict(fit, new_x)` → 여기서 `new_x`가 5행짜리이기 때문에 8-3에서 경고한 "1행일 때 상수항이 안 붙는 함정"에 걸리지 않는다. 문제에서 "5명"을 한 번에 준 것 자체가 이 함정을 피해가게 설계된 상황이라는 걸 눈치채면 좋다.
3. 예측값을 뽑아서 `new_df['sheight'] = pred['pred'].values`로 붙이는 부분 — 8-3에서 언급한 "인덱스가 정렬되어 있어서 `.values`로 붙여도 안전하다"는 조건이 그대로 성립하는 상황(순서를 안 바꿨으니까).
4. 병합 전에 `real_df['source'] = '실제 데이터'`, `new_df['source'] = '가상 데이터'`로 라벨 컬럼을 미리 추가해두는 부분 — 이걸 Mission 3만 보면 당장 필요 없어 보이지만, Mission 4에서 "색·모양으로 구분"해야 한다는 걸 미리 알고 있기 때문에 여기서 미리 준비해두는 것이다. → 문제를 풀 때 다음 미션까지 먼저 훑어보고, 지금 단계에서 나중에 필요한 자료를 미리 만들어두는 사고 습관이 여기 드러난다.

5. `concat([real_df, new_df], ignore_index=True)` — `ignore_index=True`를 쓰는 이유: 두 DataFrame을 그냥 이어 붙이면 인덱스가 0부터 각각 다시 시작해서 중복된 인덱스(0, 1, 2...가 두 번)가 생긴다. `ignore_index=True`는 합친 뒤 인덱스를 0부터 새로 순서대로 매겨줘서 이후 처리(특히 `tail()`로 마지막 5건 확인)가 꼬이지 않게 해준다.

Mission 4. 한 장의 그래프로 보고하기

문제 문장에서 잡아야 할 신호

"색·모양으로 구분", "회귀 직선을 함께 그려", "예측값이 직선 위에 정확히 놓임을 확인"

사고 흐름

1. "색·모양으로 구분"이라는 문구 → seaborn 계열 함수의 `hue`(색 구분), `markers`(모양 구분) 파라미터를 떠올려야 하는 신호. `my_plot.lmplot(..., hue='source', markers=['o', '^'])`가 그래서 나온 것.
2. "회귀 직선을 함께 그려"라는 문구 → 이걸 직접 직선을 계산해서 그리라는 게 아니라, `lmplot` 계열 함수가 산점도 위에 회귀선을 자동으로 얹어주는 기능을 가지고 있다는 걸 알고 있어야 나오는 선택이다. 새로 뭔가를 계산할 필요 없이, 이미 쓰던 시각화 함수의 기본 기능을 그대로 활용하는 것.
3. "예측값이 직선 위에 정확히 놓임을 확인" → 이게 이 미션의 진짜 목적이다. 그림을 예쁘게 그리는 게 목적이 아니라, **Mission 3에서 만든 예측값이 Mission 2에서 유도한 회귀식을 제대로 따르고 있는지 검증**하는 절차다. 만약 삼각형(가상 데이터)들이 직선에서 벗어나 있다면 → predict 단계나 회귀식 어딘가에 실수가 있었다는 신호로 읽어야 한다. → 시각화를 "결과 확인용 디버깅 도구"로도 쓴다는 관점이 여기서 중요하다.
4. `scatter_size=150`처럼 가상 데이터를 실제 데이터보다 크게 그리는 것도, "5개뿐인 예측값이 1,078개 실제 데이터에 묻히지 않고 눈에 띄게 하려는" 의도적 선택이다 — 그래프를 그릴 때 "무엇을 강조해서 보여줄 것인가"까지 미리 생각하고 파라미터를 고른다는 감각.

이 흐름 전체를 관통하는 한 문장

"문제 문장에서 동사(확인하라/유도하라/예측하라/보고하라)를 먼저 찾고, 그 동사가 요구하는 개념(EDA/모형적합/predict/시각화 검증)을 떠올린 다음, 이미 검증해둔 도구(raw 코드로 이해했던 원리, 모듈화된 함수)를 순서대로 재사용한다" — 이게 오늘 연습문제 전체를 관통하는 사고 흐름이다. 새로운 데이터셋을 받아도 이 순서(동사 파악 → 개념 매칭 → 검증된 도구 재사용)는 똑같이 적용하면 된다.

